

Cartly — Smart Trolley Intelligence Platform

Edge Demonstration Brief

ELEC0033 — UCL Year 3

Purpose of this document: Comprehensive project brief for use by an AI agent to generate a PowerPoint presentation for the edge demonstration. All technical details, marking criteria mappings, and talking points are included below.

1. Project Overview

Cartly is a real-time IoT edge intelligence platform fitted to a supermarket shopping trolley. The system captures multi-sensor telemetry, classifies shopper behaviour entirely on-device (edge), transmits data to a cloud stack, and exposes a live web dashboard with remote configuration, analytics, and live external context enrichment.

The system architecture is a three-tier edge deployment: 1. **Infrastructure layer** — Three Arduino MKR1010 boards deployed as fixed Wi-Fi access points (ANCHOR_A, ANCHOR_B, ANCHOR_C) providing the RSSI positioning grid 2. **Edge layer** — ESP32-S3 microcontroller on the trolley running all sensing, classification, actuation, and telemetry 3. **Cloud/Dashboard layer** — MQTT broker (`mqtt.yungcz.com`), FastAPI backend (`api.cartly.yungcz.com`), AWS RDS MySQL database, React frontend; enriched with live Open-Meteo weather data

1.1 Presentation Theme Guide

The PowerPoint should match the live Cartly dashboard theme so the demonstration feels like one coherent product rather than a separate academic slide deck.

Visual direction

- **Overall look:** dark, technical, minimal, high-contrast, control-room style
- **Background:** solid black `#000000` or near-black `#0a0a0a`
- **Panels/cards:** near-black surfaces with thin soft borders such as `rgba(255, 255, 255, 0.06)`
- **Primary text:** white `#FFFFFF` or off-white `#E5E7EB`
- **Secondary text:** muted slate/grey such as `#A1A1AA`, `#94A3B8`, or `#64748B`
- **Do not use:** bright academic templates, pastel colours, rounded cartoon graphics, or generic blue corporate themes

Accent colours

- **Success / live / healthy:** green #22C55E
- **Warning / queue / attention:** amber #F59E0B or #FBBF24
- **Info / cloud / analytics highlight:** blue #2563EB or #3B82F6
- **Optional premium accent for heatmap / analytics emphasis:** violet #A855F7

Use accents sparingly. Most of the slide should remain monochrome, with colour used only to direct attention to status, events, and key metrics.

Typography

- **Headings/body:** clean modern sans-serif in the style of Inter
- **Numerical values / telemetry / timestamps:** monospace in the style of JetBrains Mono
- **Heading style:** bold, compact, modern; avoid serif fonts and decorative title fonts

Layout guidance

- Use dashboard-style cards for KPIs, architecture blocks, and results
- Prefer horizontal strips, status chips, and metric tiles over bullet-heavy white slides
- Use thin dividers, subtle borders, and generous spacing
- Keep diagrams simple and sharp, with dark panels and bright labels
- Charts should use dark backgrounds with green, blue, amber, and violet series accents matching the web UI

Slide-specific styling cues

- **Title slide:** black background, centred Cartly branding, subtle blue/green glow or gradient bloom in the corners
- **Architecture slide:** dark panel blocks for Infrastructure, Edge, and Cloud layers
- **Sensor slide:** one card per sensor with white titles and muted technical annotations
- **Analytics slide:** screenshots from Metabase and the live dashboard should be framed inside dark bordered cards
- **Security / reliability slides:** use green for healthy states and amber for warnings/failsafe discussion

Tone for generated visuals

- Product-demo aesthetic rather than textbook aesthetic
- Confident and polished, similar to a startup operations dashboard
- Minimal clutter, no clip-art, no bright stock icons

2. Hardware Architecture

2.1 Infrastructure: Arduino MKR1010 Anchor Nodes

Three Arduino MKR1010 boards are deployed as static Wi-Fi access points at fixed, known positions in the store: - **ANCHOR_A** — broadcasts SSID `ANCHOR_A` - **ANCHOR_B** — broadcasts SSID `ANCHOR_B` - **ANCHOR_C** — broadcasts SSID `ANCHOR_C`

These nodes form the indoor positioning infrastructure. The ESP32-S3 on the trolley passively scans for these SSIDs and measures RSSI from each, which is then used for trilateration to estimate the trolley's 2D position within the store.

2.2 Edge Node: ESP32-S3 (Trolley-mounted)

The main processing unit. It runs the full sensor pipeline, behaviour classifier, actuator control, and communications stack.

Pins used: | Function | GPIO | |-----|-----| | IMU SDA (QMI8658) | 47 | | IMU SCL (QMI8658) | 48 | | Encoder Channel A | 1 | | Encoder Channel B | 2 | | Haptic Actuator | 21 | | LCD Backlight | 20 | | LCD DC | 38 | | LCD CS | 39 | | LCD SCK/MOSI/RST | 40/41/42 |

3. Four Sensor Types

Sensor 1: IMU Accelerometer (QMI8658 — 3-axis)

- **What it measures:** Linear acceleration in X, Y, Z axes (in mg)
- **Derived metrics:**
 - Dynamic acceleration magnitude (`aDyn`) — removes gravity component
 - Pitch and roll angles (orientation)
 - Pickup detection (threshold: 2.5 m/s²)
 - Drop/impact detection (threshold: 4.0 m/s²)
 - Step cadence estimation (from periodic acceleration peaks)
 - Vibration RMS via exponential moving average (EMA)
 - Jerk (rate of change of acceleration)
- **Update rate:** Every 20ms (50 Hz main loop)

Sensor 2: IMU Gyroscope (QMI8658 — 3-axis)

- **What it measures:** Angular velocity in X, Y, Z axes (degrees per second)
- **Derived metrics:**
 - Turn rate EMA — used for queue detection and browsing detection
 - Orientation change tracking for carry style classification
 - Yaw integration for 2D dead-reckoning position estimate

- **Same physical chip as accelerometer** (QMI8658 is a 6-axis IMU) but constitutes a distinct sensor modality

Sensor 3: RSSI / Wi-Fi Signal Scanner

- **What it measures:** Received Signal Strength Indicator (dBm) from each of the three MKR1010 anchor APs
- **Scan interval:** 15 seconds (configurable via MQTT, `rssiScanIntervalMs`)
- **Scan condition:** Only scans when speed < 0.02 m/s to avoid stalling main loop
- **Minimum detection threshold:** -120 dBm
- **Derived metrics:**
 - Zone classification: ZONE_A, ZONE_B, ZONE_C, BETWEEN (2-point fallback), UNKNOWN (<2 anchors)
 - 2D position estimate via trilateration (`posX_m`, `posY_m`)
 - `anchorsSeen` count (0–3) feeds into system health state
 - Store heatmap — Gaussian KDE over a 6×6 m floor plan rendered on the dashboard (80×80 resolution, $\sigma=3.5$ cells)

Sensor 4: Wheel Encoder (Magnetic Rotary Encoder)

- **What it measures:** Wheel rotation pulses via interrupt on GPIO 1
- **Specification:** 3 counts per wheel revolution (6-pole magnetic disc on wheel shaft, rising-edge counting on Channel A)
- **Wheel diameter:** 45.2 mm → circumference = 142.0 mm
- **Derived metrics:**
 - Distance travelled (metres)
 - Speed (m/s) — both signed (direction-aware) and absolute
 - Smoothed speed via EMA for noise rejection
- **Resolution:** ~47.3 mm per count
- **ISR protection:** Encoder count is protected via interrupt-disable critical section

4. Actuators

Actuator 1: Haptic Output (GPIO 21)

- Binary HIGH/LOW output driving a haptic element
- Non-blocking: fires for a configurable duration (`actuatorPulseMs`) then auto-releases
- **Trigger events:**
 - Pickup detected → 200 ms pulse

- Drop-down detected → 200 ms pulse
- Dwell alert (>4 s pause) → 150 ms pulse
- Queue detection → 300 ms pulse
- Can be disabled remotely via `actuatorEnabled: false` MQTT config command

Actuator 2: ST7789 TFT Display Alert (240×240)

- Inverted red/white alternating flash at 180 ms intervals
- Duration configurable via `displayFlashMs` MQTT command
- Message displayed: **"ATTENTION — Check trolley — Staff alerted"**
- Also serves as status display: shows speed, zone, carry style, system state, RSSI bars, event flags

5. Behaviour Classification (On-Device Edge Intelligence)

All classification runs locally on the ESP32-S3 — **no cloud inference required.**

Classification	Algorithm	Inputs
	5-state rule-based classifier	Speed, pitch/roll, vibration RMS, dynamic accel
	EMA of vibration + jerk + speed	Accel, speed, jerk EMA
	Threshold on <code>aDyn</code>	Accelerometer
	Threshold on impact <code>aDyn</code>	Accelerometer
	Timed state machine (speed < threshold)	Encoder speed
	Speed + turn rate + dwell combo	Encoder + gyro + dwell timer
	Creep speed + low turn + stop-go pattern + dwell	All sensors
	Peak detection on accel magnitude	Accelerometer

	RSSI trilateration (3-point, 2-point fallback)	Wi-Fi RSSI
	Gaussian KDE on 80×80 grid, rendered to canvas	Position (from RSSI trilateration)

Carry Style states: LIFTED, PARKED, TILTED_PUSH, ROUGH_ROLL, SMOOTH_ROLL

6. Power Budget

Battery Specification

- Cell type: Li-Po / Li-Ion
- Capacity: 1100 mAh
- Nominal voltage: 3.7 V
- Energy stored: $1100\text{ mAh} \times 3.7\text{ V} = 4.07\text{ Wh}$

Power Consumption

- Supply voltage: 5 V (via USB / boost converter from battery)
- Current draw (running): 0.2 A
- Active power: $5\text{ V} \times 0.2\text{ A} = 1.0\text{ W}$

Runtime Estimate

Accounting for ~87% DC-DC boost conversion efficiency:

Metric	Value
Usable energy (87% efficiency)	$4.07 \times 0.87 =$
Active runtime	$3.54\text{ Wh} \div 1.0\text{ W} =$
Typical supermarket shift	~4 hours

The 3.5-hour runtime is sufficient for a standard retail shift. The system would require charging between morning and afternoon sessions.

Power consumers (dominant): 1. Wi-Fi radio (always-on, sleep disabled for MQTT keep-alive) — est. ~150 mA 2. ST7789 TFT display + backlight — est. ~30 mA 3. ESP32-S3 core + peripherals — est. ~20 mA

4. QMI8658 IMU — est. ~1 mA

Optimisation note: Wi-Fi sleep is disabled (`WiFi.setSleep(false)`) to ensure low-latency MQTT keep-alive. Enabling Wi-Fi modem sleep (DTIM-based) could reduce power by ~30–40%, extending runtime to ~5 hours, at the cost of ~100 ms additional MQTT latency.

7. Bandwidth Analysis

Telemetry Payload

- **Format:** JSON
- **Payload size:** ~768 bytes per message
- **Fields:** 28 (device_id, timestamp, speed, distance, cadence, accel XYZ, gyro XYZ, pitch, roll, pickup, dropdown, dwell, carry_style, load_proxy, browsing, queue_detect, anchor RSSI x3, zone, posX, posY, anchors_seen, imu_ok, aws_ok, system_state, hotspot)

Upload Rate (MQTT path)

- **Upload interval:** 100 ms (`CLOUD_UPLOAD_MS = 100`)
- **Rate:** 10 messages/second
- **Upstream bandwidth:** $768\text{ B} \times 10 = 7.68\text{ KB/s} = \sim 61.4\text{ Kbps}$
- **Per hour:** ~27 MB/hour
- **Per 3.5-hour shift:** ~94 MB

Priority Event Upload

- Events (pickup, drop, queue, browsing) trigger an immediate upload with a minimum gap of 40 ms (`EVENT_UPLOAD_GAP_MS`) regardless of the regular upload interval — ensures no event is missed even at 10 Hz

Direct UDP Stream (low-latency local path)

- **Upload interval:** 50 ms (`DIRECT_UPLOAD_MS = 50`)
- **Rate:** 20 messages/second
- **Upstream bandwidth:** $768\text{ B} \times 20 = 15.4\text{ KB/s}$ (local network only)
- Opt-in only — backend must set `ENABLE_DIRECT_STREAM=1` environment variable; backend advertises its LAN IP to the device via MQTT config on connect

Bandwidth Optimisation Measures

Measure	Detail
---------	--------

	Scans every 15 s (configurable), not every loop tick — saves ~99% of Wi-Fi scan overhead
	RSSI scan only runs when speed < 0.02 m/s — avoids stalling sensor loop during motion
	Separate 40 ms gap for events vs 100 ms regular cadence — avoids flooding for non-event readings
	At-least-once delivery without QoS 2 handshake overhead
	Bypasses MQTT broker entirely on local network, halving round-trip overhead
	~35% size reduction (768 B → ~500 B) would reduce shift data to ~61 MB

8. Communications Stack

MQTT (Primary Cloud Path)

Parameter	Value
Broker	<code>mqtt.yungcz.com:8883</code>
Protocol	MQTT over TLS 1.3
Credentials	<code>cartly / cartly</code>
CA Certificate	ISRG Root X1 (Let's Encrypt), embedded in firmware
Publish topic	<code>smartrrolley/trolley_01/telemetry</code>
Subscribe topic	<code>smartrrolley/trolley_01/config</code>
QoS level	1 (at-least-once delivery)

Keep-alive	60 seconds
MQTT buffer	1280 bytes
Reconnect interval	Every 1.5 seconds on disconnect

UDP Direct Stream (Secondary Local Path)

- Port 4210, JSON over raw UDP
- Backend advertises its LAN IP to the device via MQTT config on startup
- Device switches to UDP when `directEnabled: true` is received
- 20 Hz update rate — used for lowest-latency local dashboard updates

Wi-Fi

- Connects to `YungHub` access point
- Reconnect attempt every 10 seconds on disconnect
- Sleep mode disabled to maintain MQTT responsiveness

9. Security

Implemented

Mechanism	Detail
	All MQTT traffic encrypted end-to-end between device and broker
	ISRG Root X1 embedded in firmware — prevents MITM even if a rogue CA is compromised
	Username/password required on MQTT connection
	Separate topics for telemetry (pub) and config (sub) — device cannot receive its own telemetry back
	FastAPI backend uses Pydantic models to validate all inbound config API calls

	Malformed MQTT/UDP payloads are caught and discarded without crashing the system
--	--

Why Security Is Necessary

- 1. **Config topic is an attack surface** — a malicious actor who can publish to `smartrrolley/trolley_01/config` could disable actuators, raise thresholds to suppress alerts, or redirect the direct UDP stream to a spoofed IP. TLS + authentication prevents this.
- 2. **Telemetry contains behavioural data** — shopper movement patterns, dwell times, and position data are privacy-sensitive. Encryption in transit prevents passive eavesdropping.
- 3. **Physical accessibility** — trolleys are publicly accessible; the MQTT broker being TLS-only means the credentials cannot be sniffed even on public Wi-Fi.

AWS Tools — Currently Deployed

Component	AWS Service	Status
	<code>cartly-database.czoceemm8dli.eu-west-2.rds.amazonaws.com</code> (eu-west-2)	■ Live — all telemetry stored here
	EC2 / hosted server at <code>api.cartly.yungcz.com</code>	■ Live
	Metabase on EC2 at <code>analytics.cartly.yungcz.com</code>	■ Live — directly connected to AWS RDS MySQL

AWS Tools — Proposed Additions for Full Production

Component	AWS Service	Reason
MQTT broker		Managed, scalable, integrates with Lambda/RDS
Analytics		BI dashboards directly over RDS — no extra ETL needed
File storage		Long-term CSV/parquet archival from RDS exports

Processing		Event-driven triggers on <code>queue_detect</code> / <code>pickup</code> events
Auth		Dashboard login, role-based access control
Certificates		Managed TLS certs for IoT device fleet

10. Dashboard Features

Stack: React + Recharts + WebSocket (frontend) / FastAPI + SQLAlchemy + **AWS RDS MySQL** (backend)
Frontend served from: `https://api.cartly.yungcz.com` (cloud-hosted)

Live Monitoring Panel

- Real-time speed (m/s), cadence (steps/min), distance (m), dwell time (s)
- Zone indicator (ZONE_A/B/C/BETWEEN/UNKNOWN) with colour coding
- Carry style and load proxy classification
- RSSI signal strength bars for all 3 anchors
- System state badge: `NORMAL` / `DEGRADED` / `NO_ANCHOR` / `IMU_FAIL` / `CLOUD_OFFLINE`
- Connection mode badge: `MQTT` vs `Direct UDP`
- Live event pills: Pickup (yellow), Drop-down (red), Browsing (blue), Queue (purple)

External Context Panel (Enriched Data)

Live data from **Open-Meteo Forecast API** (free, no API key) fetched every 5 minutes and displayed alongside telemetry: - **Weather:** temperature, apparent temperature, wind speed, rain (mm), cloud cover %, weather label (e.g. "Light rain", "Clear sky") - **Daylight:** sunrise, sunset, daylight hours, day/night mode - **Time context:** local time (store timezone), day of week, part-of-day label (morning rush / lunch peak / etc.), shopping period classification, weekday vs weekend - Nav bar shows live weather summary chip and shopping period chip at all times - Sources attributed: "Weather: Open-Meteo Forecast API"

Store Heatmap & Queue Panel

- **Store heatmap:** Gaussian KDE overlay on a 6×6 m SVG floor plan showing shelf rows (Bakery & Cereal, Dairy & Chilled, Fresh Produce) and 3 checkout bays (Self-Checkout, Quick Checkout, Manned Checkout)
- Positions sampled every 5th telemetry reading to build session density map
- Heatmap expandable to 500×500 px modal with colour legend (blue→cyan→yellow→red)

- **Queue status view:** automatically switches when `queue_detect` is active — highlights the nearest checkout bay with dwell time and historical per-bay visit counts

Data History & Analytics

- Speed and cadence trend line charts (120-point rolling buffer)
- Distance-over-time line chart
- Zone distribution bar chart
- Event log table: timestamp, event type, zone, speed, carry style, RSSI values
- Configurable history depth (record count selector)

Analytics Platform (Metabase)

Hosted at: <https://analytics.cartly.yungcz.com>

Metabase is the historical analytics layer over the same AWS RDS MySQL telemetry store used by the live React dashboard. This satisfies the marking requirement that stored data must connect to an analytics platform, not just appear in an operational UI.

Database connection used by Metabase: - Engine: MySQL - Host: `cartly-database.czoceemm8dli.eu-west-2.rds.amazonaws.com` - Port: `3306` - Database: `trolley` - Table analysed: `telemetry`

Recommended dashboard: Cartly Analytics

1. **Telemetry Volume Over Time** — line chart of row count by `received_at` hour; proves continuous ingestion into AWS RDS
2. **Average Speed Over Time** — line chart of average `speed_mps` by minute; shows trolley movement trends
3. **Zone Distribution** — bar chart of row count by `zone`; shows where the trolley spends most time
4. **Queue Events by Hour** — bar chart filtered to `queue_detect = true`, grouped by hour of day; highlights checkout congestion periods
5. **Pickup vs Drop-down Event Counts** — bar chart comparing counts of `pickup = true` and `drop_down = true`; quantifies trolley interaction events
6. **Browsing vs Queue Detection** — bar chart comparing counts of `browsing = true` and `queue_detect = true`; demonstrates the edge classifier outputs

Value for the demonstration: - Shows historical analysis rather than only live telemetry - Supports discussion of shopper behaviour, congestion, and zone utilisation - Strengthens the cloud story by combining EC2-hosted analytics with AWS RDS MySQL - Closes the analytics-platform gap in the marking rubric

Data Export

- **CSV export:** `GET /api/telemetry/export?format=csv` — full time-series download, all 28 columns, streamed from AWS RDS MySQL
- **JSON export:** same endpoint with `format=json`
- Streaming response (handles large datasets without loading all into memory)

Remote Configuration Panel

Six preset sensitivity profiles that push config to the device via MQTT in real-time: | Profile | Use Case |
|-----|-----| | Disabled Mode | All detection off | | Alert Mode | Full staff alerts (2 s pulse + 8 s display flash) | | Low Sensitivity | Busy store (reduced false alerts) | | High Sensitivity | Quiet store (maximum detection) | | Custom Mode | Manual parameter tuning |

Configurable parameters pushed over MQTT: - pickupAccelThreshold — acceleration threshold for pickup events - dropImpactThreshold — impact threshold for drop events - queueDwellThreshold — dwell duration to classify queue - dwellAlertThreshold — dwell duration to trigger staff alert - rssiScanIntervalMs — how often to scan for anchor APs - actuatorPulseMs — haptic pulse duration - displayFlashMs — alert display duration - actuatorEnabled — enable/disable all actuator outputs

11. System State Machine (Failsafe)

The firmware maintains a system health state that is reported in every telemetry packet and displayed on the TFT screen:

```
INIT ■■■ IMU initialisation ■■■ FAIL → IMU_FAIL (halts, displays error)
■■■ OK ■■■ WiFi + MQTT connection ■■■ Runtime state: IMU_FAIL – IMU
unavailable NO_ANCHOR – 0 anchors visible CLOUD_OFFLINE – MQTT disconnected
DEGRADED – 1 or 2 anchors (reduced positioning accuracy) NORMAL – all 3
anchors + MQTT connected
```

Automatic recovery mechanisms: - WiFi reconnect: every 10 seconds if disconnected - MQTT reconnect: every 1.5 seconds if disconnected - If MQTT unavailable, falls back to UDP direct stream (if backend previously advertised its IP) - Database: try/except with rollback on every write — a DB error never crashes the backend - Malformed MQTT/UDP packets discarded without system impact - Encoder count protected by interrupt-disable critical section (no race condition)

12. Marking Criteria Coverage Summary

Criterion	Coverage	Evidence
Key user requirements & KPIs	■	Multi-metric telemetry (speed, cadence, zone, events), behaviour classification, staff alerting
Deployment challenges	■	Battery runtime analysis, Wi-Fi coverage (3 anchors), MQTT retry logic, TLS overhead

Evidence-informed design	■	RSSI trilateration, EMA filtering, IMU dynamic acceleration gravity removal
≥4 sensor types	■	Accelerometer, Gyroscope, RSSI scanner, Wheel encoder
≥1 actuator	■	Haptic GPIO pulse + TFT display alert
Arduino MKR1010 board	■	3× MKR1010 deployed as Wi-Fi anchor APs (ANCHOR_A/B/C)
Efficient sensor aggregation	■	Single 768-byte JSON packet combines all 28 fields @ 10 Hz
Data transmitted to cloud	■	MQTT TLS to cloud broker at <code>mqtt.yungcz.com</code> , backend hosted at <code>api.cartly.yungcz.com</code>
Online database	■	AWS RDS MySQL (<code>cartly-database.czoceemm8dli.eu-west-2</code>)
Data labelled for analysis	■	28 named columns, server-side timestamp, device_id indexed
Time-series download	■	<code>/api/telemetry/export?format=csv</code> streaming endpoint
Operational dashboard	■	React live dashboard with charts, event log, zone map
Events trigger interface actions	■	Alert pills, status badge changes, real-time WebSocket push
Remote edge control	■	MQTT config topic, 6 presets + manual parameter tuning
Edge params dynamically configurable	■	All thresholds updated in real-time via MQTT, no reboot needed

Manual actuation	■	Config panel sends <code>actuatorPulseMs/</code> <code>displayFlashMs</code> commands
Automatic actuation	■	On-device pickup/drop/queue/dwell detection triggers actuator
Estimated power consumption	■	$5V \times 0.2A = 1W$; 1100mAh @ 3.7V $\rightarrow$ ~3.5 hr runtime
Bandwidth optimisation	■	100ms upload @ 768B = 61.4 Kbps; RSSI throttled to 15 s; event-priority 40ms gap; UDP opt-in local path
Wireless sensor network	■	3× MKR1010 anchor nodes + 1× ESP32-S3 edge node (star topology)
Battery powered	■	1100 mAh Li-Po @ 3.7V, 5V/0.2A draw, ~3.5 hr runtime
External data sources	■	Open-Meteo Forecast API — live weather, wind, rain, cloud, sunrise/sunset; shopping period classifier
Failsafe mechanisms	■	5-state system state machine, auto-reconnect, DB rollback, ISR protection
Analytics platform	■	Metabase deployed at <code>analytics.cartly.yungcz.com</code> connected to AWS RDS MySQL, with historical charts for speed, zones, queue events, and behaviour classification
Security protocols described	■	TLS 1.3, CA pinning, broker auth, topic isolation — all documented

AWS tools described	■	RDS MySQL deployed (eu-west-2); proposed additions: IoT Core, QuickSight, S3, Lambda, Cognito
---------------------	---	---

13. Key Talking Points for Demonstration

- End-to-end live demo flow:** Trolley moves → encoder + IMU read (50 Hz) → on-device classification → MQTT TLS publish (10 Hz) → AWS RDS MySQL write → dashboard WebSocket update in <250ms
- Edge intelligence:** All behaviour classification (carry style, queue detection, browsing) runs locally — no round-trip to cloud needed for actuation
- Dynamic reconfiguration:** Show config panel changing pickupAccelThreshold → MQTT publish → device receives and applies instantly — demonstrable by changing sensitivity and repeating a pickup gesture
- Positioning + heatmap:** Walk through 3 zones, show RSSI values and zone badge changing, then expand heatmap to show the trolley's session trail overlaid on the store floor plan
- Queue detection:** Walk to checkout area → `queue_detect` triggers → dashboard switches heatmap to Queue Status view showing the active bay and dwell time
- External context enrichment:** Point to weather panel — live Open-Meteo data (temperature, rain, cloud cover) combined with shopper telemetry; show shopping period label changing by time of day
- Battery runtime:** 1100 mAh at 3.7V → 1W draw → 3.5 hours, sufficient for a retail morning shift
- Security:** MQTT over TLS 1.3 with embedded CA cert — no cleartext credentials on wire; AWS RDS in private subnet (VPC)
- Data export:** Click export CSV — show full time-series data downloading from AWS RDS with all 28 columns labelled
- System state failsafe:** Disconnect Wi-Fi → dashboard shows `CLOUD_OFFLINE` in real time; reconnect → back to `NORMAL` within 1.5 s
- MKR1010 role:** Three Arduino MKR1010 boards act as infrastructure — fixed APs providing the RSSI grid for trilateration positioning
- Analytics platform:** Open `analytics.cartly.yungcz.com` — show Metabase charts for telemetry volume, speed trends, zone distribution, and queue-event timing to demonstrate historical cloud analytics over AWS RDS

14. Technical Stack Summary

Layer	Technology
Infrastructure nodes	Arduino MKR1010 × 3 (Wi-Fi APs)

Edge MCU	ESP32-S3
IMU	QMI8658 (6-axis, I ² C)
Display	ST7789 240×240 TFT (SPI)
Firmware language	C++ (Arduino/PlatformIO)
Cloud transport	MQTT over TLS 1.3 (PubSubClient)
Local transport	UDP JSON (port 4210)
Backend	Python FastAPI + SQLAlchemy
Database	AWS RDS MySQL (eu-west-2)
Analytics platform	Metabase (EC2-hosted, HTTPS at analytics.cartly.yungcz.com)
External data API	Open-Meteo Forecast API (weather)
MQTT client (backend)	paho-mqtt
Frontend	React + Recharts + WebSocket
Frontend host	api.cartly.yungcz.com (cloud)
Build system	PlatformIO

Document last updated: 2026-03-24. For PPT generation, map each numbered section to a slide or slide group. Sections 2–4 cover hardware, sections 5–8 cover edge intelligence and comms, sections 9–11 cover security and reliability, section 12 maps to marking rubric, section 13 provides demo script talking points.

Full-marks note: The previous analytics-platform gap is now closed by the live Metabase deployment at analytics.cartly.yungcz.com, connected directly to AWS RDS MySQL.